MATH 189 — Group 11

In [1]:
```
import os
import pandas as pd
import geopandas as gpd
import numpy as np
import re
import matplotlib.pyplot as plt
import statsmodels.api as sm
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from pathlib import Path


os.makedirs("figures", exist_ok=True)
df = pd.read_csv("ca_aqi_acs_master_county_avg.csv")
```

```
/Users/romanrespicio/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pyproj/__init__.py:96: UserWarning: Valid PROJ data directory not found. Either set the path using the environmental variable PROJ_DATA (PROJ 9.1+) | PROJ_LIB (PROJ<9.1) or with `pyproj.datadir.set_data_dir`.
  warnings.warn(str(err))
```

Problem

Problem Statement

This project investigates whether lower-income communities in California are disproportionately exposed to worse air quality, measured through AQI, PM2.5, ozone, and other pollutant indicators. By combining air quality data with socioeconomic and demographic data at the county or regional level, the project aims to determine whether income, poverty, education, unemployment, and population characteristics are statistically associated with pollution exposure. The broader goal is to identify whether patterns of air pollution in California reflect environmental inequality and whether these patterns vary across counties, pollutant types, and community characteristics.

In short, we are asking, "Are lower-income communities in California disproportionately exposed to worse air quality (identified by high AQI values and the presence of harmful pollutants such as PM2.5), and do these patterns vary by county, pollutant type, and population characteristics?"

Problem Relevance

This problem is relevant because air pollution is a major environmental and public health issue in California, and its effects may not be evenly distributed across communities. Pollutants such as PM2.5 and ozone are linked to respiratory and cardiovascular health problems, so higher exposure in lower-income areas could worsen existing social and health inequalities. By studying the relationship between socioeconomic factors and air quality, this project can help identify whether environmental risks are disproportionately concentrated in disadvantaged communities and provide insight into broader issues of environmental justice and public policy.

Data

For the data cleaning step, we combined EPA annual AQI county files from 2020 to 2024 with ACS 2024 county-level socioeconomic data. The goal was to create one clean dataset where each California county had both air quality measures and socioeconomic variables.

Data Cleaning/Transformation

The cleaning process produced two main files:

- `ca_aqi_acs_master_county_year.csv`
- `ca_aqi_acs_master_county_avg.csv`

This section imports the packages and sets paths for the raw EPA AQI files, ACS files, and output files.

In [2]:
```
scriptDir = Path(".")
inputDir = scriptDir
outputDir = scriptDir

years = range(2020, 2025)

aqiFiles = [inputDir / f"annual_aqi_by_county_{year}.csv" for year in years]
dp02File = inputDir / "ACSDP5Y2024.DP02-Data.csv"
dp03File = inputDir / "ACSDP5Y2024.DP03-Data.csv"

countyYearOut = outputDir / "ca_aqi_acs_master_county_year.csv"
countyAvgOut = outputDir / "ca_aqi_acs_master_county_avg.csv"
dataDictionaryOut = outputDir / "ca_aqi_acs_data_dictionary.csv"
missingnessOut = outputDir / "ca_aqi_acs_missingness_report.csv"
```

We kept AQI variables that describe overall air quality, unhealthy air quality days, and the pollutant most responsible for AQI on each day.

From ACS, we kept income, poverty, unemployment, education, and health insurance variables because these directly connect to socioeconomic status.

In [3]:
```
aqiRename = {
    "State": "state",
    "County": "county",
    "Year": "year",
    "Days with AQI": "days_with_aqi",
    "Good Days": "good_days",
    "Moderate Days": "moderate_days",
    "Unhealthy for Sensitive Groups Days": "usg_days",
    "Unhealthy Days": "unhealthy_days",
    "Very Unhealthy Days": "very_unhealthy_days",
    "Hazardous Days": "hazardous_days",
    "Max AQI": "max_aqi",
    "90th Percentile AQI": "aqi_90th_percentile",
    "Median AQI": "median_aqi",
    "Days CO": "days_co_main_pollutant",
    "Days NO2": "days_no2_main_pollutant",
    "Days Ozone": "days_ozone_main_pollutant",
    "Days PM2.5": "days_pm25_main_pollutant",
    "Days PM10": "days_pm10_main_pollutant",
}

dp02Cols = {
    "GEO_ID": "geo_id",
    "NAME": "acs_name",
    "DP02_0059E": "population_25_plus",
    "DP02_0067PE": "pct_high_school_or_higher",
    "DP02_0068PE": "pct_bachelors_or_higher",
}

dp03Cols = {
    "GEO_ID": "geo_id",
    "NAME": "acs_name",
    "DP03_0001E": "population_16_plus",
    "DP03_0009PE": "unemployment_rate",
    "DP03_0062E": "median_household_income",
    "DP03_0063E": "mean_household_income",
    "DP03_0088E": "per_capita_income",
    "DP03_0128PE": "poverty_rate_all_people",
    "DP03_0099PE": "pct_no_health_insurance",
}
```

These helper functions standardize county names and convert ACS values into numeric form. This was needed because EPA and ACS county names had slightly different formatting, and some ACS numeric values were stored as strings.

In [4]:
```
def cleanCountyName(name):
    if pd.isna(name):
        return np.nan

    cleaned = str(name).strip()
    cleaned = re.sub(r",\s*California$", "", cleaned)
```

```
    cleaned = re.sub(r"\s+County$", "", cleaned)
    cleaned = re.sub(r"\s+", " ", cleaned)
    return cleaned


def makeNumeric(series):
    cleaned = series.astype(str).str.replace(",", "", regex=False)
    cleaned = cleaned.replace({"nan": np.nan, "N": np.nan, "-": np.nan})
    cleaned = cleaned.replace({"(X)": np.nan})
    cleaned = cleaned.str.replace("+", "", regex=False)
    cleaned = cleaned.str.replace("median-", "", regex=False)
    cleaned = cleaned.str.replace("median+", "", regex=False)
    return pd.to_numeric(cleaned, errors="coerce")
```

The AQI files were originally separated by year, so we combined the 2020 - 2024 files into one dataset. We filtered to California counties, cleaned cuonty names, converted numeric columns, and created new variables for bad AQI days and pollutant-day percentages.

In [5]:
```
def loadAqiData():
    frames = []

    for path in aqiFiles:
        frames.append(pd.read_csv(path))

    aqi = pd.concat(frames, ignore_index=True)
    aqi = aqi.rename(columns=aqiRename)
    aqi = aqi[aqi["state"].eq("California")].copy()
    aqi["county"] = aqi["county"].apply(cleanCountyName)

    numericCols = [col for col in aqi.columns if col not in {"state", "count
    for col in numericCols:
        aqi[col] = makeNumeric(aqi[col])

    aqi["bad_aqi_days"] = (
        aqi["usg_days"]
        + aqi["unhealthy_days"]
        + aqi["very_unhealthy_days"]
        + aqi["hazardous_days"]
    )

    aqi["bad_aqi_day_pct"] = np.where(
        aqi["days_with_aqi"] > 0,
        aqi["bad_aqi_days"] / aqi["days_with_aqi"] * 100,
        np.nan,
    )

    pollutantDayCols = [
        "days_co_main_pollutant",
        "days_no2_main_pollutant",
        "days_ozone_main_pollutant",
        "days_pm25_main_pollutant",
        "days_pm10_main_pollutant",
    ]
```

```
    for col in pollutantDayCols:
        pctCol = "pct_" + col
        aqi[pctCol] = np.where(
            aqi["days_with_aqi"] > 0,
            aqi[col] / aqi["days_with_aqi"] * 100,
            np.nan,
        )

    return aqi.sort_values(["county", "year"]).reset_index(drop=True)
```

The ACS data was split across DP02 and DP03. DP02 provided education and social variables, while DP03 provided income, poverty, unemployment, and health insurance variables. We filtered to California counties, cleaned county names, converted numeric values, and merged the two ACS tables together.

In [6]:
```
def loadAcsTable(path, keepCols):
    acs = pd.read_csv(path, dtype=str)
    acs = acs[acs["GEO_ID"].str.startswith("0500000US06", na=False)].copy()
    acs = acs[list(keepCols)].rename(columns=keepCols)
    acs["county"] = acs["acs_name"].apply(cleanCountyName)

    for col in acs.columns:
        if col not in {"geo_id", "acs_name", "county"}:
            acs[col] = makeNumeric(acs[col])

    return acs


def loadAcsData():
    dp02 = loadAcsTable(dp02File, dp02Cols)
    dp03 = loadAcsTable(dp03File, dp03Cols)

    acs = dp02.merge(
        dp03.drop(columns=["acs_name"]),
        on=["geo_id", "county"],
        how="outer",
        validate="one_to_one",
    )

    acs["income_quartile"] = pd.qcut(
        acs["median_household_income"],
        q=4,
        labels=["Q1 lowest income", "Q2", "Q3", "Q4 highest income"],
    )

    incomeCutoff = acs["median_household_income"].median()
    acs["lower_income_group"] = acs["median_household_income"] <= incomeCutc

    return acs.sort_values("county").reset_index(drop=True)
```

After cleaning both datasets, we merged AQI and ACS data by county. This created the main county-year dataset. We also created a county-average dataset by averaging AQI variables from 2020 - 2024 while keeping ACS variables fixed.

In [7]:
```
aqi = loadAqiData()
acs = loadAcsData()

masterByYear = aqi.merge(acs, on="county", how="left", validate="many_to_one

masterByYear.shape
masterByYear.head()
```

Out[7]:

	state	county	year	days_with_aqi	good_days	moderate_days	usg_days	u
0	California	Alameda	2020	366	169	177	8	
1	California	Alameda	2021	365	155	200	9	
2	California	Alameda	2022	365	185	175	5	
3	California	Alameda	2023	365	197	155	13	
4	California	Alameda	2024	366	194	171	1	

5 rows × 39 columns

The county-average dataset gives one row per county. This is useful for correlation analysis because the ACS socioeconomic variables repeat across years, while the AQI values change from 2020 to 2024.

To make this file, we averaged the AQI variables across the five years and kept the ACS variables fixed for each county.

In [8]:
```
def averageByCounty(master):
    aqiCols = [
        "days_with_aqi",
        "good_days",
        "moderate_days",
        "usg_days",
        "unhealthy_days",
        "very_unhealthy_days",
        "hazardous_days",
        "bad_aqi_days",
        "bad_aqi_day_pct",
        "max_aqi",
        "aqi_90th_percentile",
        "median_aqi",
        "days_co_main_pollutant",
        "days_no2_main_pollutant",
        "days_ozone_main_pollutant",
        "days_pm25_main_pollutant",
        "days_pm10_main_pollutant",
        "pct_days_co_main_pollutant",
```

```
        "pct_days_no2_main_pollutant",
        "pct_days_ozone_main_pollutant",
        "pct_days_pm25_main_pollutant",
        "pct_days_pm10_main_pollutant",
    ]

    acsCols = [
        "geo_id",
        "acs_name",
        "population_25_plus",
        "pct_high_school_or_higher",
        "pct_bachelors_or_higher",
        "population_16_plus",
        "unemployment_rate",
        "median_household_income",
        "mean_household_income",
        "per_capita_income",
        "poverty_rate_all_people",
        "pct_no_health_insurance",
        "income_quartile",
        "lower_income_group",
    ]

    countyAqi = (
        master.groupby("county", as_index=False)[aqiCols]
        .mean(numeric_only=True)
        .rename(columns={col: f"avg_{col}_2020_2024" for col in aqiCols})
    )

    acsByCounty = master[["county"] + acsCols].drop_duplicates("county")
    return acsByCounty.merge(countyAqi, on="county", how="left")

masterCountyAvg = averageByCounty(masterByYear)

masterCountyAvg.shape
masterCountyAvg.head()
```

Out[8]:

	county	geo_id	acs_name	population_25_plus	pct_high_school_or_hi
0	Alameda	0500000US06001	Alameda County, California	1192437	
1	Amador	0500000US06005	Amador County, California	32875	
2	Butte	0500000US06007	Butte County, California	135188	
3	Calaveras	0500000US06009	Calaveras County, California	35917	
4	Colusa	0500000US06011	Colusa County, California	14077	

5 rows × 37 columns

The cleaned data was saved into four output files. The county-year file keeps each county and year separately, while the county-average file is better for correlation analysis because it has one row per county.

In [9]:
```
masterByYear.to_csv(countyYearOut, index=False)
masterCountyAvg.to_csv(countyAvgOut, index=False)

print("Saved cleaned files:")
print(f"- {countyYearOut}")
print(f"- {countyAvgOut}")
```

```
Saved cleaned files:
- ca_aqi_acs_master_county_year.csv
- ca_aqi_acs_master_county_avg.csv
```

Exploratory Data Analysis (EDA)

Visual Analyses

We conducted exploratory data analysis (EDA) to better understand the distributions, relationships, and geographic patterns present in the dataset before building predictive models. Through summary statistics and visualizations, we identified key trends linking air quality measures across California communities.

In [10]:
```
counties = gpd.read_file(
    "https://www2.census.gov/geo/tiger/GENZ2023/shp/cb_2023_us_county_500k.z
```

```
)

ca = counties[counties["STATE_NAME"] == "California"].copy()

csv_url = "ca_aqi_acs_master_county_avg.csv"
df = pd.read_csv(csv_url)

df["county_clean"] = df["county"].str.lower().str.strip()
ca["county_clean"] = ca["NAME"].str.lower().str.strip()

merged = ca.merge(df, on="county_clean", how="left")

fig, ax = plt.subplots(figsize=(10, 12))

merged.plot(
    column="avg_bad_aqi_day_pct_2020_2024",
    cmap="Reds",
    linewidth=0.5,
    edgecolor="black",
    legend=True,
    ax=ax,
    missing_kwds={"color": "lightgrey", "label": "Missing data"}
)

ax.set_title(
    "Average Percentage of Unhealthy AQI Days by California County (2020–202
    fontsize=14
)

ax.axis("off")
plt.tight_layout()

output_path = "california_bad_aqi_map.png"
plt.savefig(output_path, dpi=300, bbox_inches="tight")

plt.show()

print(f"Saved map to: {output_path}")

top10 = (
    df.sort_values("avg_bad_aqi_day_pct_2020_2024", ascending=False)
    [["county", "avg_bad_aqi_day_pct_2020_2024"]]
    .head(10)
)

print("\nTop 10 Counties by Bad AQI Percentage:")
print(top10.to_string(index=False))
```

```
---------------------------------------------------------------------------
DataDirError                              Traceback (most recent call last)
Cell In[10], line 1
----> 1 counties = gpd.read_file(
      2 
      3 )
      5 ca = counties[counties["STATE_NAME"] == "California"].copy()
      7 csv_url = "ca_aqi_acs_master_county_avg.csv"

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/io/file.py:316, in _read_file(filename, bbox, mask, columns, rows, engin
e, **kwargs)
    313             filename = response.read()
    315 if engine == "pyogrio":
--> 316     return _read_file_pyogrio(
    317         filename, bbox=bbox, mask=mask, columns=columns, rows=rows,
**kwargs
    318     )
    320 elif engine == "fiona":
    321     if pd.api.types.is_file_like(filename):

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/io/file.py:576, in _read_file_pyogrio(path_or_bytes, bbox, mask, rows, *
*kwargs)
    567     warnings.warn(
    568         "The 'include_fields' and 'ignore_fields' keywords are depre
cated, and "
    569         "will be removed in a future release. You can use the 'colum
ns' keyword "
   (...)    572         stacklevel=3,
    573     )
    574     kwargs["columns"] = kwargs.pop("include_fields")
--> 576 return pyogrio.read_dataframe(path_or_bytes, bbox=bbox, **kwargs)

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pyogri
o/geopandas.py:525, in read_dataframe(path_or_buffer, layer, encoding, colum
ns, read_geometry, force_2d, skip_features, max_features, where, bbox, mask,
fids, sql, sql_dialect, fid_as_index, use_arrow, on_invalid, arrow_to_pandas
_kwargs, datetime_as_string, mixed_offsets_as_utc, **kwargs)
    521     return df
    523 geometry = shapely.from_wkb(geometry, on_invalid=on_invalid)
--> 525 return gp.GeoDataFrame(df, geometry=geometry, crs=meta[     ])

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/geodataframe.py:243, in GeoDataFrame.__init__(self, data, geometry, crs,
*args, **kwargs)
    235     if isinstance(geometry, pd.Series) and geometry.name not in (
    236         "geometry",
    237         None,
    238     ):
    239         # __init__ always creates geometry col named "geometry"
    240         # rename as `set_geometry` respects the given series name
    241         geometry = geometry.rename("geometry")
--> 243     self.set_geometry(geometry, inplace=True, crs=crs)
    245 if geometry is None and crs:
```

```
    246     raise ValueError(
    247         "Assigning CRS to a GeoDataFrame without a geometry column i
s not "
    248         "supported. Supply geometry using the 'geometry=' keyword ar
gument, "
    249         "or by providing a DataFrame with column name 'geometry'",
    250     )

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/geodataframe.py:464, in GeoDataFrame.set_geometry(self, col, drop, inpla
ce, crs)
    461     crs = getattr(level, "crs", None)
    463 # Check that we are using a listlike of geometries
--> 464 level = _ensure_geometry(level, crs=crs)
    465 # ensure_geometry only sets crs on level if it has crs==None
    466 if isinstance(level, GeoSeries):

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/geodataframe.py:71, in _ensure_geometry(data, crs)
     69     return GeoSeries(out, index=data.index, name=data.name)
     70 else:
---> 71     out = from_shapely(data, crs=crs)
     72     return out

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/array.py:190, in from_shapely(data, crs)
    187             raise TypeError(f"Input must be valid geometry objects:
{geom}")
    188     arr = np.array(out, dtype=object)
--> 190 return GeometryArray(arr, crs=crs)

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/array.py:344, in GeometryArray.__init__(self, data, crs)
    341 self._data = data
    343 self._crs = None
--> 344 self.crs = crs
    345 self._sindex = None

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/geopan
das/array.py:395, in GeometryArray.crs(self, value)
    392 if HAS_PYPROJ:
    393     from pyproj import CRS
--> 395     self._crs = None if not value else CRS.from_user_input(value)
    396 else:
    397     if value is not None:

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pypro
j/crs/crs.py:503, in CRS.from_user_input(cls, value, **kwargs)
    501 if isinstance(value, cls):
    502     return value
--> 503 return cls(value, **kwargs)

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pypro
j/crs/crs.py:350, in CRS.__init__(self, projparams, **kwargs)
    348     self._local.crs = projparams
    349 else:
```

```
--> 350     self._local.crs = _CRS(self.srs)

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pypro
j/_crs.pyx:2356, in pyproj._crs._CRS.__init__()

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pypro
j/_context.pyx:214, in pyproj._context.pyproj_context_create()

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pypro
j/_context.pyx:163, in pyproj._context.pyproj_context_initialize()

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pypro
j/_context.pyx:136, in pyproj._context.set_context_data_dir()

File ~/Downloads/cse152a_wi26_hw0/.conda/lib/python3.11/site-packages/pypro
j/datadir.py:114, in get_data_dir()
    111             _VALIDATED_PROJ_DATA = str(system_proj_dir)
    113 if _VALIDATED_PROJ_DATA is None:
--> 114     raise DataDirError(
    115         "Valid PROJ data directory not found. "
    116         "Either set the path using the environmental variable "
    117         "PROJ_DATA (PROJ 9.1+) | PROJ_LIB (PROJ<9.1) or "
    118         "with `pyproj.datadir.set_data_dir`."
    119     )
    120 return _VALIDATED_PROJ_DATA

DataDirError: Valid PROJ data directory not found. Either set the path using
the environmental variable PROJ_DATA (PROJ 9.1+) | PROJ_LIB (PROJ<9.1) or wi
th `pyproj.datadir.set_data_dir`.
```

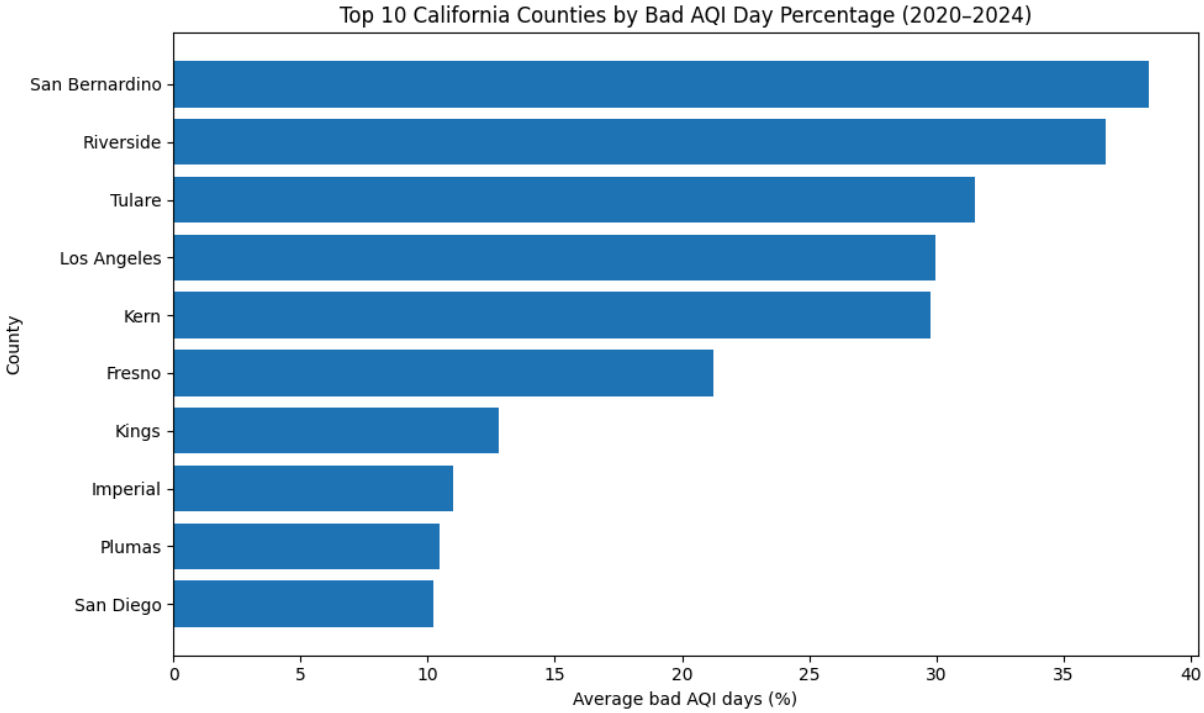
We performed a geographic visualization analysis using county-level air quality data from California. A choropleth map was created to display the average percentage of unhealthy AQI days between 2020 and 2024 across California counties. Counties were shaded according to their pollution exposure levels, allowing us to identify patterns and pollution hotspots and patterns throughout the state.

The map revealed a lot of geographic variation in air quality across California. Counties with the highest percentages of unhealthy AQI days included San Bernardino, Riverside, Tulare, Los Angeles, and Kern. In contrast, many coastal and northern counties had lower levels of pollution exposure. These findings suggest that air quality burdens are not evenly distributed across California and that certain regions experience greater pollution exposure than others.

This analysis was conducted at the county level, which could hide important differences within counties. Large counties can contain communities with very different pollution exposures, but these local variations are not captured by county averages. In addition, some counties with high pollution levels also contain large urban areas, such as Los Angeles County. Higher pollution in these counties may be influenced by factors such as population density, traffic, and industrial activity rather than socioeconomic characteristics alone. As a result, the map may not be able to confidently determine the specific causes of poor air quality. Finally, the analysis relies on AQI summary measures

rather than direct pollutant concentration measurements, which might not fully capture all aspects of pollution exposure.

In [11]:
```
#2
metric = "avg_bad_aqi_day_pct_2020_2024"

top10 = (
    df.sort_values(metric, ascending=False)
    [["county", metric]]
    .head(10)
    .sort_values(metric, ascending=True)
)

fig, ax = plt.subplots(figsize=(10, 6))

ax.barh(top10["county"], top10[metric])

ax.set_title("Top 10 California Counties by Bad AQI Day Percentage (2020–202
ax.set_xlabel("Average bad AQI days (%)")
ax.set_ylabel("County")

plt.tight_layout()

output_path = "figures/top10_bad_aqi_counties.png"
plt.savefig(output_path, dpi=300, bbox_inches="tight")

# show image in notebook
plt.show()

plt.close()

print(f"Saved bar chart to: {output_path}")
print("\nTop 10 counties:")
print(top10.sort_values(metric, ascending=False).to_string(index=False))
```




```
Saved bar chart to: figures/top10_bad_aqi_counties.png

Top 10 counties:
        county  avg_bad_aqi_day_pct_2020_2024
San Bernardino                      38.364997
     Riverside                      36.668613
        Tulare                      31.524815
   Los Angeles                      29.986975
          Kern                      29.774684
        Fresno                      21.233625
         Kings                      12.806198
      Imperial                      11.002021
        Plumas                      10.473173
     San Diego                      10.231454
```

We created a horizontal bar chart showing the ten California counties with the highest average percentage of unhealthy AQI days between 2020 and 2024. The counties were ranked from highest to lowest pollution exposure to allow for direct comparison of the areas most affected by poor air quality.

The bar chart helped us identify San Bernardino and Riverside counties as having the highest percentages of unhealthy AQI days, followed by Tulare, Los Angeles, Kern, and Fresno. These results suggest that pollution exposure is concentrated in a relatively small number of counties rather than being evenly distributed throughout California. The chart also supports the patterns observed in the choropleth map by highlighting the counties that consistently experience the worst air quality.

This visualization only includes the ten counties with the highest pollution exposure and therefore does not provide a complete picture of air quality across all California counties. In addition, the chart is based on county-level averages, which may hide variation within individual counties. As mentioned earlier, some counties on the list also contain large

urban areas with higher population density, traffic, and industrial activity, all of which can contribute to poorer air quality. Because of this, the chart cannot determine why certain counties experience worse air quality and should be interpreted as a descriptive summary rather than reporting of a causal relationship.

Correlational Analysis

Correlation Analysis Setup

This section sets up the files used for the correlation analysis. The analysis uses the cleaned county-year dataset, the county-average dataset, the data dictionary, and the missingness report created during data cleaning.

The county-average dataset is the main dataset for correlation analysis because it gives one row per county. This avoids treating repeated ACS socioeconomic values across multiple years as separate observations.

In [12]:
```
outputDir = Path(".")

countyYearFile = Path("ca_aqi_acs_master_county_year.csv")
countyAvgFile  = Path("ca_aqi_acs_master_county_avg.csv")
dataDictFile   = Path("ca_aqi_acs_data_dictionary.csv")
missingFile    = Path("ca_aqi_acs_missingness_report.csv")

allCorrOut = outputDir / "correlation_results_all.csv"
keyCorrOut = outputDir / "correlation_results_key_pairs.csv"
matrixOut  = outputDir / "correlation_matrix_soc_air.csv"
yearlyOut  = outputDir / "year_by_year_correlation_summary.csv"
summaryOut = outputDir / "correlation_analysis_summary.md"
heatmapOut = outputDir / "figures/correlation_heatmap_soc_air.png"
```

The socioeconomic variables were selected because they describe income, poverty, employment, education, and health access across California counties. These are the main community-level factors related to our environmental inequality question.

The air quality variables were selected because they describe both general AQI conditions and worse pollution conditions. We also included pollutant main-driver percentages for PM2.5, ozone, and NO2. These are not actual concentration values, but they show how often each pollutant was the main driver of AQI.

In [13]:
```
socioCols = [
    "median_household_income",
    "mean_household_income",
    "per_capita_income",
    "poverty_rate_all_people",
    "unemployment_rate",
    "pct_high_school_or_higher",
    "pct_bachelors_or_higher",
    "pct_no_health_insurance",
]
```

```
airCols = [
    "avg_bad_aqi_day_pct_2020_2024",
    "avg_bad_aqi_days_2020_2024",
    "avg_median_aqi_2020_2024",
    "avg_aqi_90th_percentile_2020_2024",
    "avg_max_aqi_2020_2024",
    "avg_pct_days_pm25_main_pollutant_2020_2024",
    "avg_pct_days_ozone_main_pollutant_2020_2024",
    "avg_pct_days_no2_main_pollutant_2020_2024",
]

yearlyAirMap = {
    "avg_bad_aqi_day_pct_2020_2024": "bad_aqi_day_pct",
    "avg_median_aqi_2020_2024": "median_aqi",
    "avg_aqi_90th_percentile_2020_2024": "aqi_90th_percentile",
    "avg_max_aqi_2020_2024": "max_aqi",
    "avg_pct_days_pm25_main_pollutant_2020_2024": "pct_days_pm25_main_pollut
    "avg_pct_days_ozone_main_pollutant_2020_2024": "pct_days_ozone_main_poll
}
```

This is just assigning prettier names and labels for the files later

In [14]:
```
prettyNames = {
    "median_household_income": "median household income",
    "mean_household_income": "mean household income",
    "per_capita_income": "per capita income",
    "poverty_rate_all_people": "poverty rate",
    "unemployment_rate": "unemployment rate",
    "pct_high_school_or_higher": "high school or higher",
    "pct_bachelors_or_higher": "bachelor's or higher",
    "pct_no_health_insurance": "no health insurance",
    "avg_bad_aqi_day_pct_2020_2024": "bad AQI day %",
    "avg_bad_aqi_days_2020_2024": "bad AQI days",
    "avg_median_aqi_2020_2024": "median AQI",
    "avg_aqi_90th_percentile_2020_2024": "90th percentile AQI",
    "avg_max_aqi_2020_2024": "max AQI",
    "avg_pct_days_pm25_main_pollutant_2020_2024": "PM2.5 main-pollutant day
    "avg_pct_days_ozone_main_pollutant_2020_2024": "ozone main-pollutant day
    "avg_pct_days_no2_main_pollutant_2020_2024": "NO2 main-pollutant day %",
}
```

The main helper function calculates Pearson and Spearman correlations for each pair of variables. Pearson correlation measures linear association, while Spearman correlation checks whether two variables generally move together by rank.

The script also includes a p-value adjustment function because many correlations are tested at once. This reduces the chance of overemphasizing a result that only looks significant because many tests were run.

In [15]:
```
def getCorr(data, xCol, yCol):
    pairData = data[[xCol, yCol]].dropna()
    n = len(pairData)
```

```
    if n < 3:
        return None
    pearsonR, pearsonP = stats.pearsonr(pairData[xCol], pairData[yCol])
    spearmanR, spearmanP = stats.spearmanr(pairData[xCol], pairData[yCol])
    return {
        "socio_variable": xCol,
        "air_variable": yCol,
        "n": n,
        "pearson_r": pearsonR,
        "pearson_p": pearsonP,
        "spearman_r": spearmanR,
        "spearman_p": spearmanP,
    }


def addBenjaminiHochberg(results, pCol, outCol):
    ranked = results[pCol].rank(method="first")
    m = results[pCol].notna().sum()
    results[outCol] = (results[pCol] * m / ranked).clip(upper=1)
    results[outCol] = results[outCol].sort_values(ascending=False).cummin()
    return results
```

This function makes scatterplots for selected socioeconomic and air quality variables. Each plot includes a fitted line, Pearson correlation value, and p-value so the relationship is easier to interpret visually.

In [16]:
```
def saveScatter(countyAvg, xCol, yCol, fileName):
    plotData = countyAvg[["county", xCol, yCol]].dropna()
    slope, intercept, rVal, pVal, stdErr = stats.linregress(plotData[xCol],
    fig, ax = plt.subplots(figsize=(7, 5))
    ax.scatter(plotData[xCol], plotData[yCol])
    xLine = np.linspace(plotData[xCol].min(), plotData[xCol].max(), 100)
    yLine = intercept + slope * xLine
    ax.plot(xLine, yLine)
    ax.set_xlabel(prettyNames.get(xCol, xCol))
    ax.set_ylabel(prettyNames.get(yCol, yCol))
    ax.set_title(f"{prettyNames.get(xCol, xCol)} vs {prettyNames.get(yCol, y
    ax.text(0.03, 0.97, f"r = {rVal:.2f}\np = {pVal:.4f}", transform=ax.trar
    fig.tight_layout()
    fig.savefig(outputDir / fileName, dpi=300)
    plt.close(fig)
```

Laod cleaned data from earlier

In [17]:
```
neededFiles = [countyYearFile, countyAvgFile, dataDictFile, missingFile]
missingFiles = [path for path in neededFiles if not path.exists()]
if missingFiles:
    raise FileNotFoundError(f"Missing cleaned files: {missingFiles}")

countyYear    = pd.read_csv(countyYearFile)
countyAvg     = pd.read_csv(countyAvgFile)
dataDictionary = pd.read_csv(dataDictFile)
missingReport = pd.read_csv(missingFile)
```

Check for any missing column values and such

In [18]:
```
highMissing = missingReport[missingReport["missing_pct"] > 20]
if len(highMissing) > 0:
    print("Warning: some columns have high missingness:")
    print(highMissing[["column", "missing_pct"]])
```

This section calculates Pearson and Spearman correlations between each socioeconomic variable and each air quality variable. Pearson measures linear relationships, while Spearman checks whether the variables generally move together by rank.

In [19]:
```
rows = []
for socioCol in socioCols:
    for airCol in airCols:
        corrRow = getCorr(countyAvg, socioCol, airCol)
        if corrRow is not None:
            rows.append(corrRow)

results = pd.DataFrame(rows)
results["abs_pearson_r"] = results["pearson_r"].abs()
results["abs_spearman_r"] = results["spearman_r"].abs()
results = addBenjaminiHochberg(results, "pearson_p", "pearson_fdr_p")
results = addBenjaminiHochberg(results, "spearman_p", "spearman_fdr_p")
results = results.sort_values("abs_pearson_r", ascending=False)
```

This section keeps the air quality variables most directly related to our project question, including bad AQI day percentage, median AQI, 90th percentile AQI, and PM2.5/ozone main-pollutant day percentages.

In [20]:
```
keyPairs = results[
    results["air_variable"].isin([
        "avg_bad_aqi_day_pct_2020_2024",
        "avg_median_aqi_2020_2024",
        "avg_aqi_90th_percentile_2020_2024",
        "avg_pct_days_pm25_main_pollutant_2020_2024",
        "avg_pct_days_ozone_main_pollutant_2020_2024",
    ])
].copy()
```

Create the m,atrix of Pearson correlations.

In [21]:
```
matrix = pd.DataFrame(index=socioCols, columns=airCols, dtype=float)
for socioCol in socioCols:
    for airCol in airCols:
        corrRow = getCorr(countyAvg, socioCol, airCol)
        if corrRow is not None:
            matrix.loc[socioCol, airCol] = corrRow["pearson_r"]
```

Save correlation resultsa s a csv file

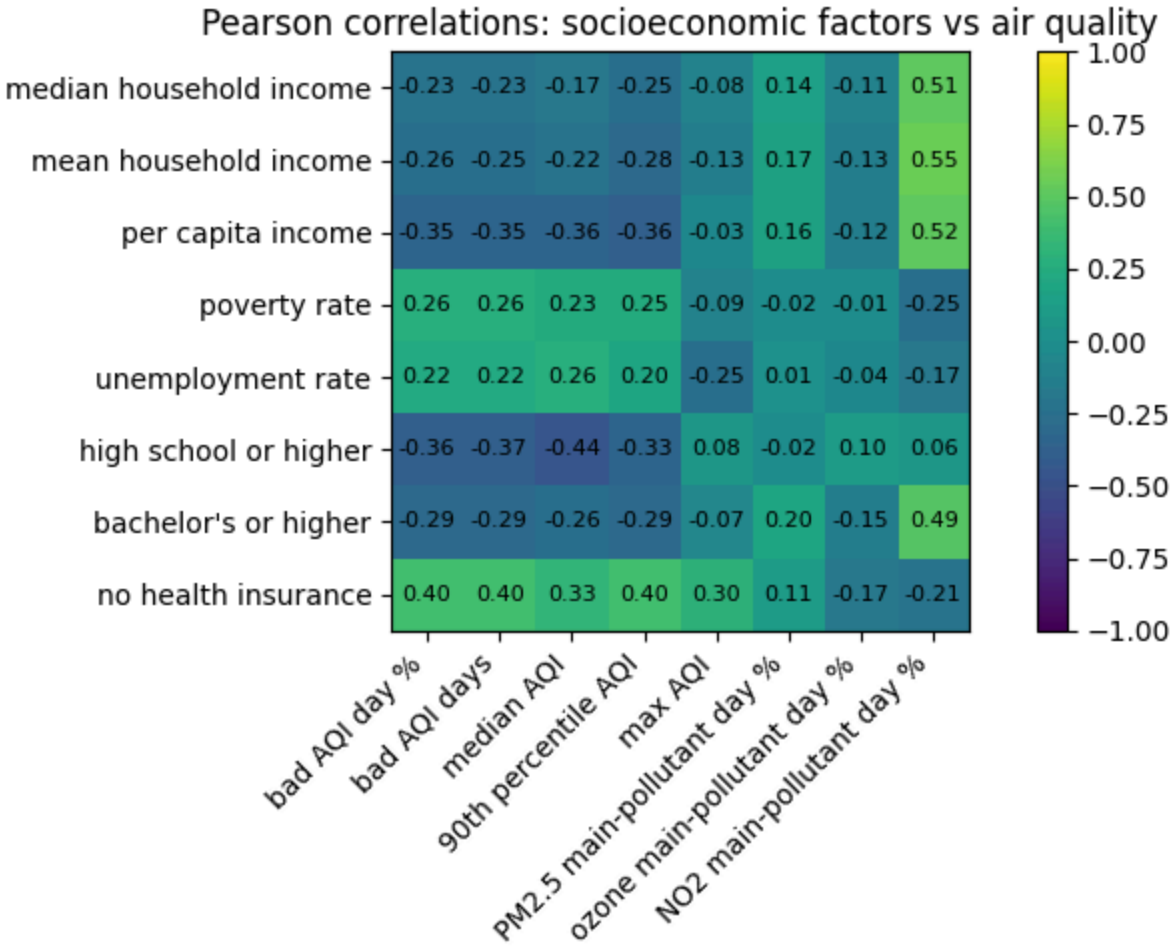
In [22]:
```
results.to_csv(allCorrOut, index=False)
keyPairs.to_csv(keyCorrOut, index=False)
matrix.to_csv(matrixOut)
```

The main analysis uses county averages, but this section checks each AQI year separately. This is only a sensitivity check because the ACS socioeconomic values repeat across years.

In [23]:
```
rows = []
for year, yearData in countyYear.groupby("year"):
    for socioCol in socioCols:
        for avgCol, yearCol in yearlyAirMap.items():
            corrRow = getCorr(yearData, socioCol, yearCol)
            if corrRow is not None:
                corrRow["year"] = int(year)
                corrRow["air_variable"] = yearCol
                rows.append(corrRow)

yearly = pd.DataFrame(rows)
yearly = yearly.sort_values(["year", "pearson_p"])
yearly.to_csv(yearlyOut, index=False)
```

Create the correlation heatmap to show the Pearson corerlations between socioeconomic variables and air quality variables.

In [24]:
```
plotMatrix = matrix.copy()
plotMatrix.index = [prettyNames.get(col, col) for col in plotMatrix.index]
plotMatrix.columns = [prettyNames.get(col, col) for col in plotMatrix.columr

figHeight = max(5, len(plotMatrix.index) * 0.55)
figWidth = max(8, len(plotMatrix.columns) * 1.15)
fig, ax = plt.subplots(figsize=(figWidth, figHeight))
image = ax.imshow(plotMatrix.values.astype(float), vmin=-1, vmax=1)
ax.set_xticks(np.arange(len(plotMatrix.columns)))
ax.set_yticks(np.arange(len(plotMatrix.index)))
ax.set_xticklabels(plotMatrix.columns, rotation=45, ha="right")
ax.set_yticklabels(plotMatrix.index)
for i in range(len(plotMatrix.index)):
    for j in range(len(plotMatrix.columns)):
        value = plotMatrix.iloc[i, j]
        if pd.notna(value):
            ax.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize
ax.set_title("Pearson correlations: socioeconomic factors vs air quality")
fig.colorbar(image, ax=ax, fraction=0.035, pad=0.04)
fig.tight_layout()
fig.savefig(heatmapOut, dpi=300)
plt.show()
```



These scatterplots show a few important relationships more directly. They help check whether the correlation results match visible patterns in the data.

In [25]:
```
saveScatter(countyAvg, "median_household_income", "avg_bad_aqi_day_pct_2020_
saveScatter(countyAvg, "poverty_rate_all_people",  "avg_bad_aqi_day_pct_2020
saveScatter(countyAvg, "pct_no_health_insurance",  "avg_bad_aqi_day_pct_2020
```

Final results showing

In [26]:
```
colsToShow = ["socio_variable", "air_variable", "n", "pearson_r", "pearson_p
print(keyPairs.sort_values("pearson_p")[colsToShow].head(8).round(4))
```

```
                socio_variable                        air_variable   n  \
42  pct_high_school_or_higher           avg_median_aqi_2020_2024  53
56    pct_no_health_insurance      avg_bad_aqi_day_pct_2020_2024  53
59    pct_no_health_insurance  avg_aqi_90th_percentile_2020_2024  53
40  pct_high_school_or_higher      avg_bad_aqi_day_pct_2020_2024  53
19          per_capita_income  avg_aqi_90th_percentile_2020_2024  53
18          per_capita_income           avg_median_aqi_2020_2024  53
16          per_capita_income      avg_bad_aqi_day_pct_2020_2024  53
58    pct_no_health_insurance           avg_median_aqi_2020_2024  53

    pearson_r  pearson_p  spearman_r  spearman_p
42    -0.4422     0.0009     -0.3764      0.0055
56     0.4032     0.0028      0.4392      0.0010
59     0.3976     0.0032      0.4104      0.0023
40    -0.3641     0.0074     -0.2660      0.0542
19    -0.3638     0.0074     -0.3746      0.0057
18    -0.3592     0.0083     -0.3474      0.0108
16    -0.3480     0.0107     -0.4539      0.0006
58     0.3342     0.0145      0.3127      0.0226
```

Overall, the correlation analysis gives some spuport for the project question. Several socioeconomic variables move in the expected direction. Counties with higher income or higher education tend to have better AQI outcomes, while counties with more uninsured residents tend to have worse AQI outcomes.

However, most correlations are weak to moderate. This means the results show patterns, not proof of causation. The correlation analysis is useful as an early statistical step, but it should be followed by regression, hypothesis testing, and mapping before making stronger conclusions.

One limitation is that the pollutant variables are not actual PM2.5 or ozone concentration measurements. They show the percent of days where each pollutant was the main driver of AQI. Because of this, the pollutant results should be described as main-pollutant day patterns, not direct concentration analysis.

Statistical Analyses (Clustering, Hypothesis Testing, Linear Regression)

Clustering

We applied K-means clustering to identify groups of California counties with similar socioeconomic and environmental characteristics.

Six variables were included in the analysis:

Bachelor's degree attainment rate Unemployment rate Median household income Poverty rate Percentage of bad AQI days Median AQI

Before clustering, all variables were standardized using z-score normalization to account for differences in scale.

To determine the number of clusters, the Elbow Method was applied. The elbow plot suggested that K = 3 provides a reasonable balance between model complexity and explanatory power.

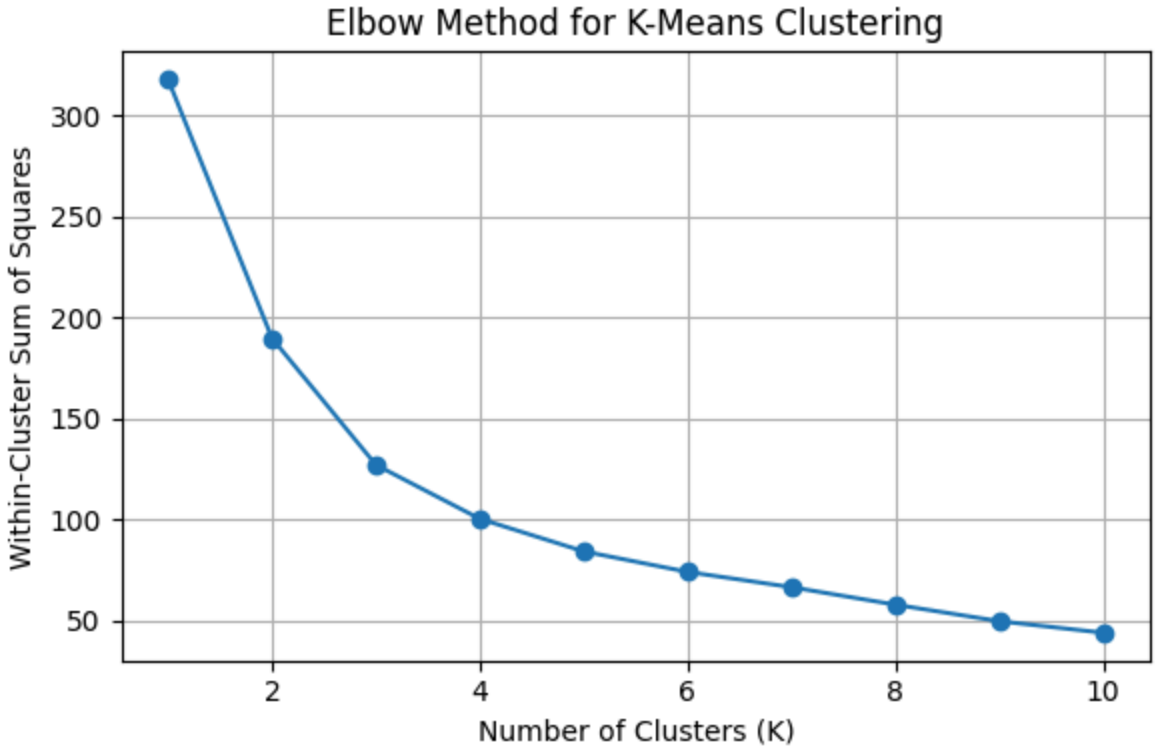
The clustering results identified three distinct county groups. One cluster, consisting of Fresno, Kern, Los Angeles, Riverside, San Bernardino, and Tulare, exhibited substantially worse air quality outcomes than the other counties.

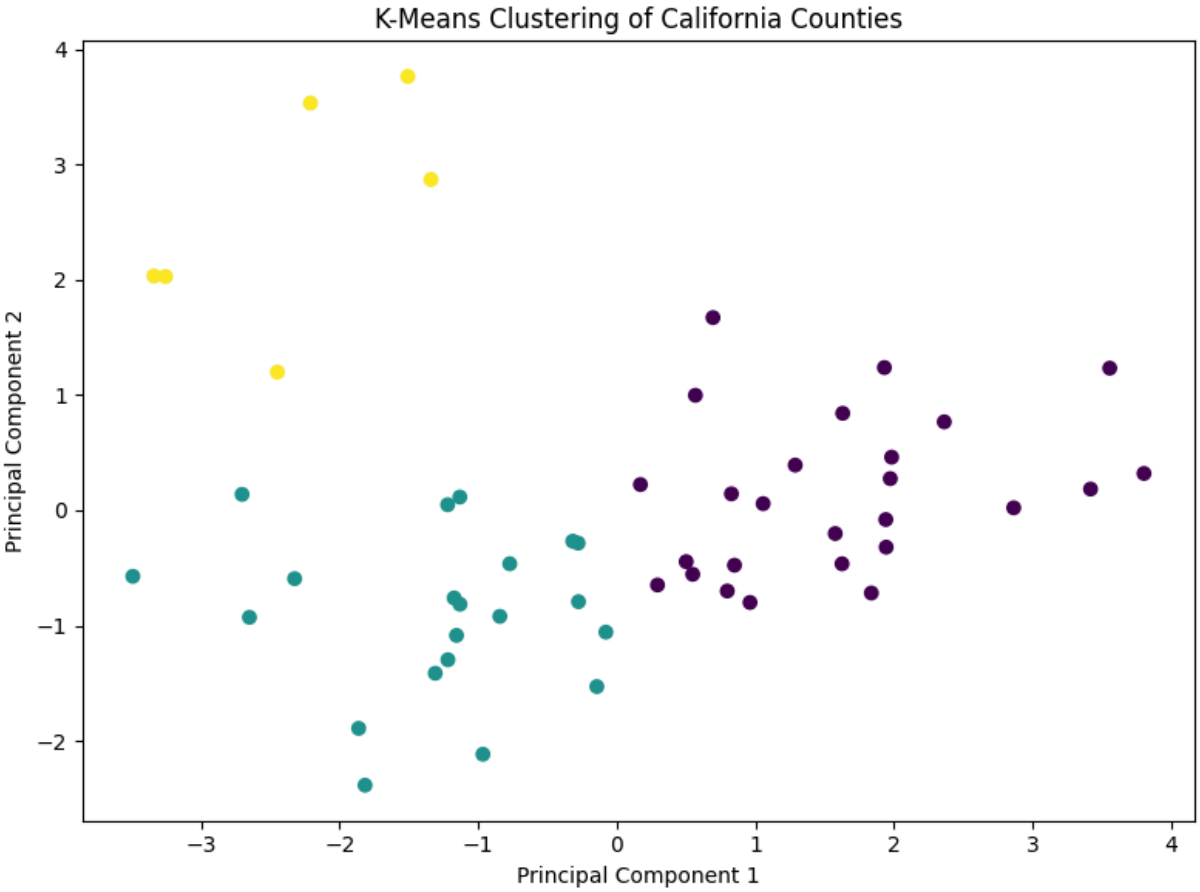
To visualize the results, Principal Component Analysis (PCA) was used to project the six-dimensional data into two dimensions. The PCA plot showed clear separation among the three clusters, supporting the presence of distinct socioeconomic and environmental county profiles.

In [27]:
```
features = [
    "pct_bachelors_or_higher",
    "unemployment_rate",
    "median_household_income",
    "poverty_rate_all_people",
    "avg_bad_aqi_day_pct_2020_2024",
    "avg_median_aqi_2020_2024"
]

X = df[features]
X_scaled = StandardScaler().fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X_scaled)

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 1000)

print("\nCluster counts:")
print(df["cluster"].value_counts())

print("\nCluster means:")
print(df.groupby("cluster")[features].mean().round(2))

print("\nCounties in Cluster 2:")
print(df[df["cluster"] == 2][["county"]])

# Figure 1: Elbow plot
k_values = range(1, 11)
inertia = []

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia.append(km.inertia_)
```

```
plt.figure(1, figsize=(6, 4))
plt.plot(k_values, inertia, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Within-Cluster Sum of Squares")
plt.title("Elbow Method for K-Means Clustering")
plt.grid(True)
plt.tight_layout()
plt.savefig("figures/elbow_plot.png")
plt.show()

# Figure 2: PCA plot
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(2, figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df["cluster"])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering of California Counties")
plt.tight_layout()
plt.savefig("figures/pca_clusters.png")
plt.show()
```

```
Cluster counts:
cluster
0    26
1    21
2     6
Name: count, dtype: int64

Cluster means:
         pct_bachelors_or_higher  unemployment_rate  median_household_income
poverty_rate_all_people  avg_bad_aqi_day_pct_2020_2024  avg_median_aqi_2020_
2024
cluster
0                          40.23               5.38                110606.38
10.07                           3.23                     44.83
1                          21.65               8.55                 69152.29
15.51                           5.02                     47.10
2                          24.10               7.92                 80729.17
15.48                          31.26                     77.70

Counties in Cluster 2:
            county
8           Fresno
13            Kern
16     Los Angeles
29       Riverside
32  San Bernardino
49          Tulare
```





Results

The K-means algorithm identified three distinct clusters of California counties.

Cluster 0 contained 26 counties and was characterized by higher educational attainment, higher household income, lower poverty rates, and generally better air quality outcomes. Cluster 1 contained 21 counties and exhibited lower income and education levels, higher poverty and unemployment rates, and moderate air quality outcomes. Cluster 2 contained only 6 counties: Fresno, Kern, Los Angeles, Riverside, San Bernardino, and Tulare.

Cluster 2 exhibited substantially worse environmental conditions than the other two clusters. On average, counties in this cluster experienced 31.3% bad AQI days and a median AQI of 77.7, compared with approximately 3–5% bad AQI days and median AQI values below 50 in the other clusters.

The PCA visualization showed clear separation among the three clusters, supporting the existence of distinct county groups based on socioeconomic and environmental characteristics.

Interpretation

The clustering results suggest that California counties do not form a single continuum of socioeconomic and environmental conditions. Instead, they can be grouped into distinct profiles.

Most notably, the six counties in Cluster 2 experienced substantially higher pollution burdens than the rest of the state. These counties also tended to have lower educational attainment, higher poverty rates, and higher unemployment levels than counties in Cluster 0.

This finding supports the broader conclusion of the project that environmental burdens are associated with multiple socioeconomic disadvantages rather than any single factor alone. While income by itself was not always a significant predictor of air quality outcomes, considering several socioeconomic variables simultaneously reveals meaningful patterns across California counties.

Hypothesis Testing

We tested whether lower income California counties experience statistically worse air quality than higher income counties. Counties were split using the `lower_income_group` marker in the dataset, placing the bottom two income quartiles in the "lower-income group" and the top two in the "higher-income group".

We tested four air quality outcomes: Median AQI, percentage of bad AQI days, percentage of days where PM2.5 was the main pollutant, and percentage of days where ozone was the main pollutant.

For each outcome:

Null hypothesis (H_0): The mean air quality outcome is the same in lower-income and higher-income counties.

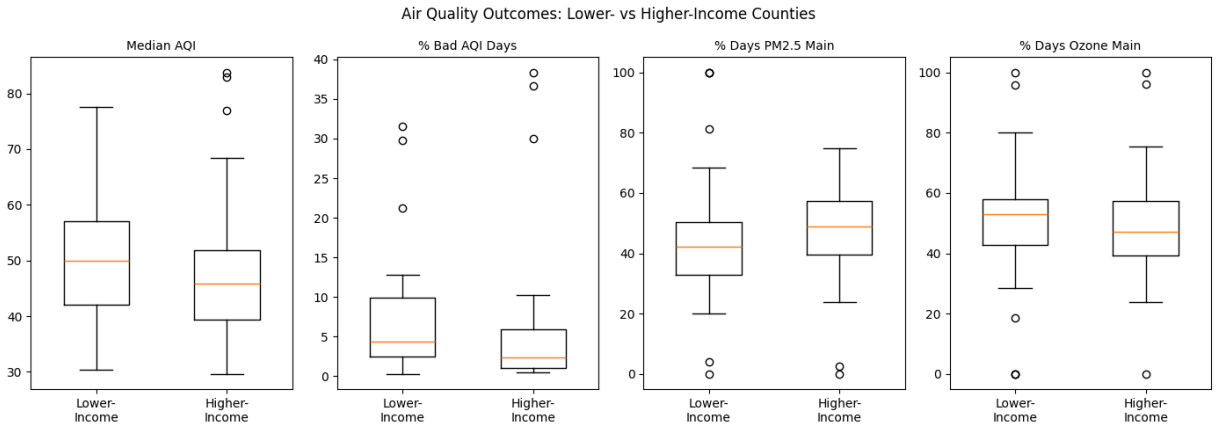
Alternative hypothesis (H_a): Lower-income counties have a worse (higher) air quality outcome than higher-income counties.

Test statistic: Welch t-statistic (one-sided), confirmed with a Mann-Whitney U test. Significance level α = 0.05.

In [28]:
```
low  = df[df["lower_income_group"] == True]
high = df[df["lower_income_group"] == False]

outcomes = {
    "avg_median_aqi_2020_2024":                    "Median AQI",
    "avg_bad_aqi_day_pct_2020_2024":               "% Bad AQI Days",
    "avg_pct_days_pm25_main_pollutant_2020_2024":  "% Days PM2.5 Main",
    "avg_pct_days_ozone_main_pollutant_2020_2024": "% Days Ozone Main",
}

rows = []
for col, label in outcomes.items():
    x_low  = low[col].dropna()
    x_high = high[col].dropna()

    t_stat, t_p_two = stats.ttest_ind(x_low, x_high, equal_var=False)
    t_p_one = t_p_two / 2 if t_stat > 0 else 1 - t_p_two / 2

    _, mwu_p = stats.mannwhitneyu(x_low, x_high, alternative="greater")

    pooled_std = np.sqrt((x_low.std(ddof=1)**2 + x_high.std(ddof=1)**2) / 2)
    d = (x_low.mean() - x_high.mean()) / pooled_std

    rows.append({
        "Outcome":        label,
        "Mean (Low-Inc)": round(x_low.mean(), 3),
        "Mean (Hi-Inc)":  round(x_high.mean(), 3),
        "t-stat":         round(t_stat, 4),
        "p (1-tail)": round(t_p_one, 4),
        "MWU p":          round(mwu_p, 4),
        "Cohen's d":      round(d, 3),
        "Sig (a=0.05)":   "Yes" if t_p_one < 0.05 else "No",
    })

results = pd.DataFrame(rows)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 1000)
print(results.to_string(index=False))
```

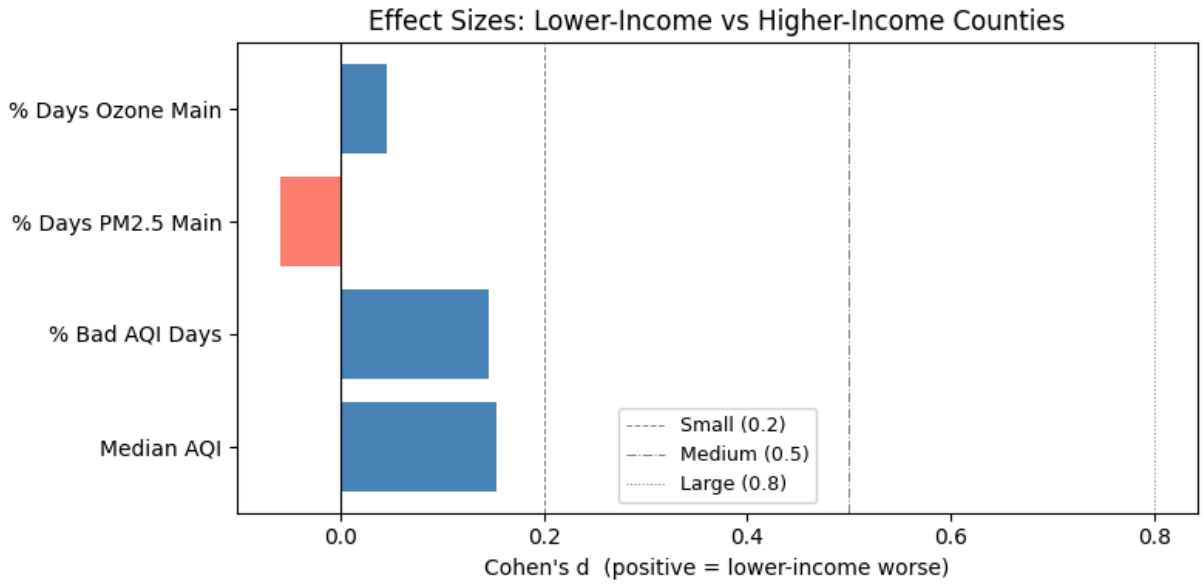
```
          Outcome  Mean (Low-Inc)  Mean (Hi-Inc)  t-stat  p (1-tail)  MWU p
Cohen's d Sig (a=0.05)
       Median AQI          50.544         48.479  0.5565      0.2901 0.1702
0.153           No
   % Bad AQI Days           7.838          6.464  0.5311      0.2989 0.0367
0.145           No
% Days PM2.5 Main          45.936         47.268 -0.2112      0.5831 0.8939
-0.059           No
% Days Ozone Main          50.288         49.234  0.1630      0.4356 0.2520
0.045           No
```

In [29]:
```
fig, axes = plt.subplots(1, len(outcomes), figsize=(14, 5))

for ax, (col, label) in zip(axes, outcomes.items()):
    data = [low[col].dropna().values, high[col].dropna().values]
    ax.boxplot(data, tick_labels=["Lower-\nIncome", "Higher-\nIncome"], widt
    ax.set_title(label, fontsize=10)

plt.suptitle("Air Quality Outcomes: Lower- vs Higher-Income Counties", fonts
plt.tight_layout()
plt.savefig("figures/hypothesis_boxplots.png")
plt.show()
```



Cohen's d

In [30]:
```
fig, ax = plt.subplots(figsize=(8, 4))
labels   = [r["Outcome"] for r in rows]
d_values = [r["Cohen's d"] for r in rows]
colors   = ["steelblue" if d > 0 else "salmon" for d in d_values]
ax.barh(labels, d_values, color=colors)
ax.axvline(0,   color="black", linewidth=0.8)
ax.axvline(0.2, color="gray",  linestyle="--", linewidth=0.7, label="Small (
ax.axvline(0.5, color="gray",  linestyle="-.", linewidth=0.7, label="Medium
ax.axvline(0.8, color="gray",  linestyle=":",  linewidth=0.7, label="Large (
ax.set_xlabel("Cohen's d  (positive = lower-income worse)")
ax.set_title("Effect Sizes: Lower-Income vs Higher-Income Counties")
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig("figures/hypothesis_effect_sizes.png")
plt.show()
```



Results

None of the four outcomes produced a statistically significant difference at α = 0.05. For Median AQI, the lower income group averaged 50.5 versus 48.5 for higher income counties, but the Welch t-test returned t = 0.557 and p = 0.290, with a Cohen's d of 0.153. For percentage of bad AQI days, the means were 7.8 versus 6.5, with t = 0.531, p = 0.299, and d = 0.145. For percentage of days where PM2.5 was the main pollutant, lower income counties actually averaged slightly lower (45.9 vs 47.3), producing t = -0.211 and a negative d of -0.059. For ozone, the difference was also not significant. (t = 0.163, p = 0.436).

The Mann-Whitney U test on percentage of bad AQI days returned p = 0.037, which crosses the threshold in isolation, but does not hold up alongside the other three non-significant results.

Interpretation

At the county level, income group alone **does not** clearly distinguish worse or better air quality. Cohen's d values are all below 0.2, meaning the directional differences are small in practical terms. Some of California's highest income counties sit geographically close to other heavily polluted areas, which may affect any income based signal when averaged at the county scale.

Linear Regression

We fit multiple linear regression models to estimate how well socioeconomic variables predict air quality outcomes across 53 California counties. The five predictors were median household income, poverty rate, unemployment rate, percentage with a bachelor's degree or higher, and percentage with no health insurance. All predictors were standardized before fitting so their coefficients are directly comparable in magnitude.

Three separate OLS models were fit, one each for Median AQI, percentage of bad AQI days, and percentage of days where PM2.5 was the main pollutant.

Null hypothesis (H_0): The socioeconomic predictors have no linear relationship with the air quality outcome (all slope coefficients equal zero).

Alternative hypothesis (H_a): At least one socioeconomic predictor has a nonzero linear relationship with the air quality outcome.

Test statistic: F-statistic from the omnibus F-test. Individual predictors are evaluated with t-statistics on each coefficient. Significance level $\alpha = 0.05$.

In [31]:
```
predictors = [
    "median_household_income",
    "poverty_rate_all_people",
    "unemployment_rate",
    "pct_bachelors_or_higher",
    "pct_no_health_insurance",
]

reg_outcomes = {
    "avg_median_aqi_2020_2024":                    "Median AQI",
    "avg_bad_aqi_day_pct_2020_2024":               "% Bad AQI Days",
    "avg_pct_days_pm25_main_pollutant_2020_2024": "% Days PM2.5 Main",
}

reg_df   = df[predictors + list(reg_outcomes.keys())].dropna().copy()
X_scaled = StandardScaler().fit_transform(reg_df[predictors])
X_df     = pd.DataFrame(X_scaled, columns=predictors, index=reg_df.index)
X        = sm.add_constant(X_df)

models = {}
for col, label in reg_outcomes.items():
    model = sm.OLS(reg_df[col], X).fit()
    models[col] = model
    print(f"\n{'='*55}")
    print(f"Outcome: {label}")
    print(f"  R²      = {model.rsquared:.4f}")
    print(f"  Adj. R² = {model.rsquared_adj:.4f}")
    print(f"  F-stat  = {model.fvalue:.4f}  (p = {model.f_pvalue:.4f})")
    print(model.summary())
```

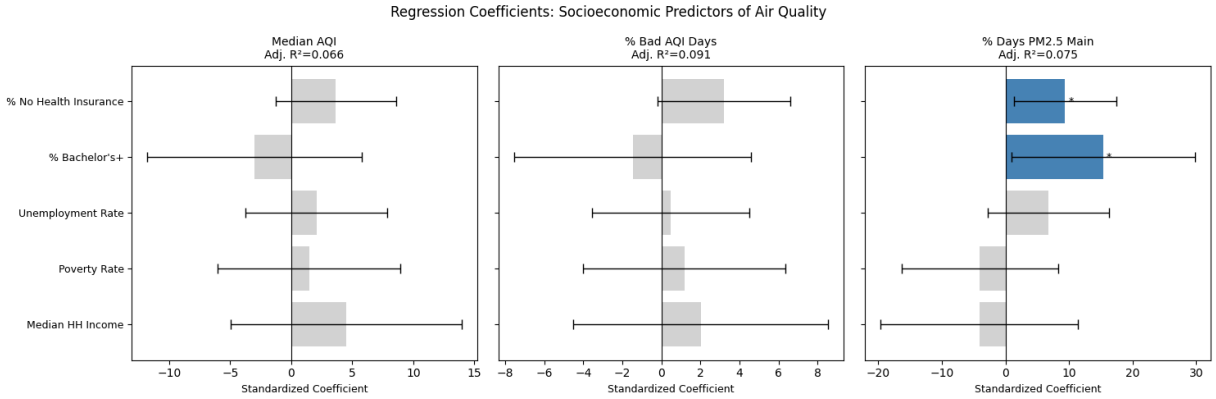
```
======================================================
Outcome: Median AQI
  R²      = 0.1561
  Adj. R² = 0.0663
  F-stat  = 1.7387  (p = 0.1443)
                            OLS Regression Results
==============================================================================
========
Dep. Variable:     avg_median_aqi_2020_2024   R-squared:
0.156
Model:                                  OLS   Adj. R-squared:
0.066
Method:                       Least Squares   F-statistic:
1.739
Date:                      Sun, 07 Jun 2026   Prob (F-statistic):
0.144
Time:                              23:47:32   Log-Likelihood:
-208.08
No. Observations:                        53   AIC:
428.2
Df Residuals:                            47   BIC:
440.0
Df Model:                                 5
Covariance Type:                  nonrobust
==============================================================================
===============
                              coef    std err          t      P>|t|      [0.
025      0.975]
------------------------------------------------------------------------------
---------------
const                       49.4528      1.790     27.633      0.000      45.
853      53.053
median_household_income      4.4972      4.696      0.958      0.343      -4.
951      13.945
poverty_rate_all_people      1.4630      3.726      0.393      0.696      -6.
032       8.958
unemployment_rate            2.0640      2.894      0.713      0.479      -3.
758       7.886
pct_bachelors_or_higher     -2.9961      4.364     -0.687      0.496     -11.
775       5.783
pct_no_health_insurance      3.6350      2.451      1.483      0.145      -1.
295       8.565
==============================================================================
==
Omnibus:                        2.187   Durbin-Watson:                   1.8
43
Prob(Omnibus):                  0.335   Jarque-Bera (JB):                1.8
59
Skew:                           0.457   Prob(JB):                        0.3
95
Kurtosis:                       2.910   Cond. No.                         7.
11
==============================================================================
==

Notes:
```

```
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.

=====================================================
Outcome: % Bad AQI Days
  R²      = 0.1782
  Adj. R² = 0.0907
  F-stat  = 2.0379  (p = 0.0904)
                                OLS Regression Results
==============================================================================
=============
Dep. Variable:     avg_bad_aqi_day_pct_2020_2024   R-squared:
0.178
Model:                                       OLS   Adj. R-squared:
0.091
Method:                            Least Squares   F-statistic:
2.038
Date:                           Sun, 07 Jun 2026   Prob (F-statistic):
0.0904
Time:                                   23:47:32   Log-Likelihood:
-188.56
No. Observations:                             53   AIC:
389.1
Df Residuals:                                 47   BIC:
400.9
Df Model:                                      5
Covariance Type:                       nonrobust
==============================================================================
===============
                               coef    std err          t      P>|t|      [0.
025      0.975]
------------------------------------------------------------------------------
---------------
const                        7.1124      1.238      5.744      0.000       4.
621       9.604
median_household_income      2.0106      3.250      0.619      0.539      -4.
527       8.548
poverty_rate_all_people      1.1771      2.578      0.457      0.650      -4.
009       6.363
unemployment_rate            0.4671      2.002      0.233      0.817      -3.
561       4.496
pct_bachelors_or_higher     -1.4749      3.020     -0.488      0.628      -7.
549       4.600
pct_no_health_insurance      3.2098      1.696      1.893      0.065      -0.
202       6.621
==============================================================================
==
Omnibus:                       22.600   Durbin-Watson:                   1.9
97
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               32.4
82
Skew:                           1.521   Prob(JB):                     8.84e-
08
Kurtosis:                       5.335   Cond. No.                        7.
11
==============================================================================
```

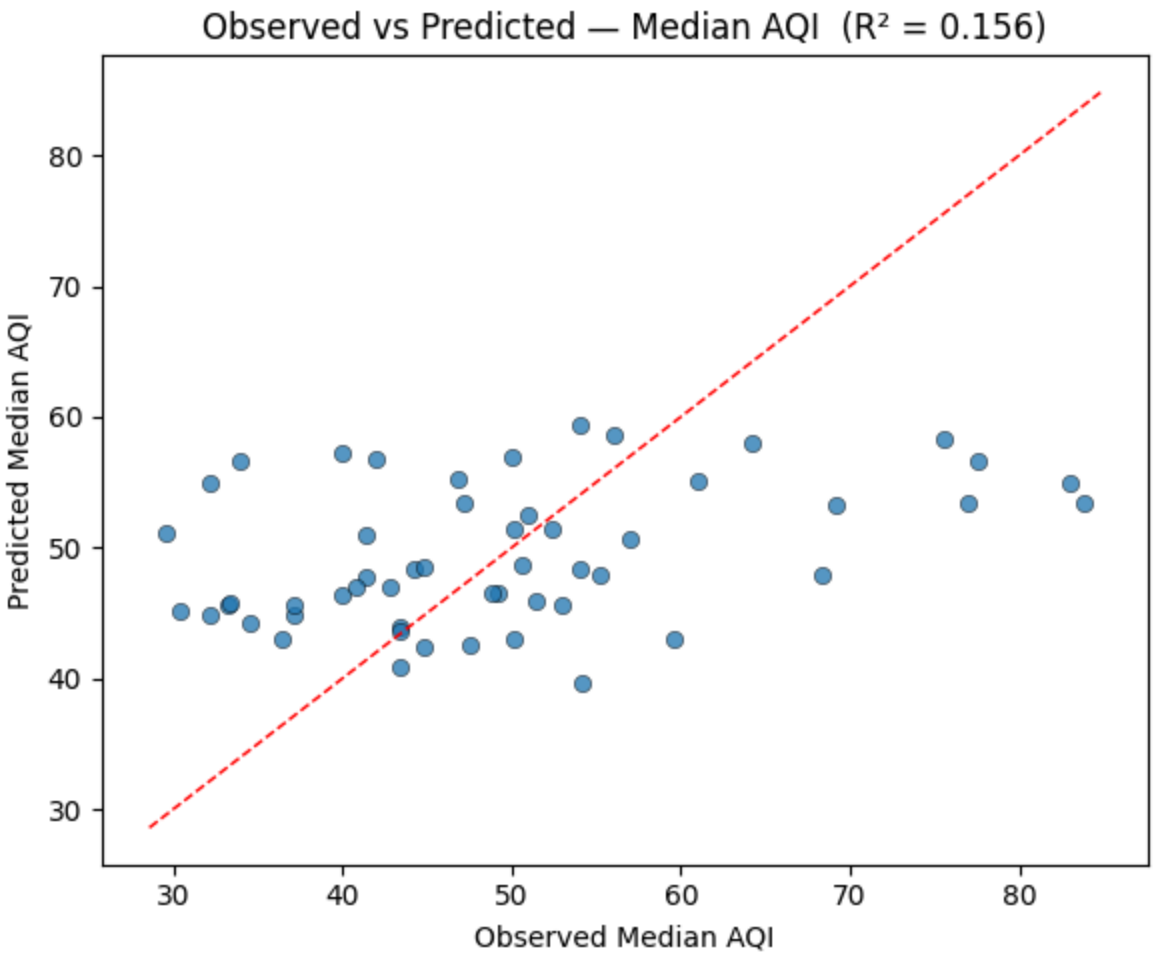
```
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.

=====================================================
Outcome: % Days PM2.5 Main
  R²      = 0.1637
  Adj. R² = 0.0747
  F-stat  = 1.8399  (p = 0.1233)
                                    OLS Regression Results
==============================================================================
==========================
Dep. Variable:     avg_pct_days_pm25_main_pollutant_2020_2024   R-squared:
0.164
Model:                                                    OLS   Adj. R-squar
ed:                  0.075
Method:                                     Least Squares   F-statistic:
1.840
Date:                                    Sun, 07 Jun 2026   Prob (F-stat
istic):              0.123
Time:                                           23:47:32   Log-Likeliho
od:                -234.37
No. Observations:                                     53   AIC:
480.7
Df Residuals:                                         47   BIC:
492.6
Df Model:                                              5
Covariance Type:                                nonrobust
==============================================================================
===============
                               coef    std err          t      P>|t|      [0.
025      0.975]
------------------------------------------------------------------------------
---------------
const                       46.6399      2.939     15.871      0.000      40.
728      52.552
median_household_income     -4.0944      7.712     -0.531      0.598     -19.
609      11.420
poverty_rate_all_people     -4.0270      6.118     -0.658      0.514     -16.
334       8.281
unemployment_rate            6.7991      4.752      1.431      0.159      -2.
761      16.359
pct_bachelors_or_higher     15.4021      7.166      2.149      0.037       0.
986      29.818
pct_no_health_insurance      9.3799      4.024      2.331      0.024       1.
284      17.476
==============================================================================
==
Omnibus:                       10.935   Durbin-Watson:                   2.0
67
Prob(Omnibus):                  0.004   Jarque-Bera (JB):               13.5
30
Skew:                           0.740   Prob(JB):                     0.001
15
```

```
Kurtosis:                       4.984   Cond. No.                         7.
11
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
```

In [32]:
```
pred_labels = {
    "median_household_income": "Median HH Income",
    "poverty_rate_all_people": "Poverty Rate",
    "unemployment_rate":       "Unemployment Rate",
    "pct_bachelors_or_higher": "% Bachelor's+",
    "pct_no_health_insurance": "% No Health Insurance",
}

# Figure 3: Coefficient plot
fig, axes = plt.subplots(1, 3, figsize=(15, 5), sharey=True)

for ax, (col, label) in zip(axes, reg_outcomes.items()):
    model = models[col]
    coef  = model.params.drop("const")
    conf  = model.conf_int().drop("const")
    pvals = model.pvalues.drop("const")

    y_pos  = np.arange(len(coef))
    colors = ["steelblue" if p < 0.05 else "lightgray" for p in pvals]

    ax.barh(y_pos, coef.values, color=colors,
            xerr=[coef.values - conf[0].values, conf[1].values - coef.values
            capsize=4, error_kw={"linewidth": 1})
    ax.axvline(0, color="black", linewidth=0.8)
    ax.set_yticks(y_pos)
    ax.set_yticklabels([pred_labels[v] for v in predictors], fontsize=9)
    ax.set_title(f"{label}\nAdj. R²={model.rsquared_adj:.3f}", fontsize=10)
    ax.set_xlabel("Standardized Coefficient", fontsize=9)

    for i, (c, p) in enumerate(zip(coef.values, pvals)):
        star = "***" if p < 0.001 else ("**" if p < 0.01 else ("*" if p < 0.
        if star:
            ax.text(c + (0.5 if c >= 0 else -0.5), i, star,
                    va="center", ha="left" if c >= 0 else "right", fontsize=

plt.suptitle("Regression Coefficients: Socioeconomic Predictors of Air Quali
plt.tight_layout()
plt.savefig("figures/regression_coefficients.png")
plt.show()
```



In [33]:
```
# Figure 4: Observed vs Predicted (Median AQI)
primary = models["avg_median_aqi_2020_2024"]
fitted  = primary.fittedvalues
y_true  = reg_df["avg_median_aqi_2020_2024"]

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(y_true, fitted, alpha=0.75, edgecolors="k", linewidths=0.4)
lim = [min(y_true.min(), fitted.min()) - 1, max(y_true.max(), fitted.max())
ax.plot(lim, lim, "r--", linewidth=1)
ax.set_xlabel("Observed Median AQI")
ax.set_ylabel("Predicted Median AQI")
ax.set_title(f"Observed vs Predicted — Median AQI  (R² = {primary.rsquared:.
plt.tight_layout()
plt.savefig("figures/regression_observed_vs_predicted.png")
plt.show()
```



Results

For Median AQI, the model produced an R^2 of 0.156 and an adjusted R^2 of 0.069. The F-statistic was 1.74 with $p = 0.144$, meaning the predictors as a set did not significantly explain variation in median AQI. No individual predictor reached significance, though percentage with no health insurance had the largest positive coefficient (3.64).

For percentage of bad AQI days, the model explained slightly more variance with $R^2 = 0.178$ and adjusted $R^2 = 0.094$. The F-test returned $F = 2.04$ with $p = 0.090$, again non-significant. Percentage with no health insurance had the largest coefficient (3.21, $p = 0.065$), narrowly missing significance.

For percentage of days where PM2.5 was the main pollutant, R^2 was 0.164 with adjusted R^2 of 0.076 and $F = 1.84$ with $p = 0.123$. Two predictors reached significance at $\alpha = 0.05$: percentage with a bachelor's degree or higher ($\beta = 15.40$, $t = 2.15$, $p = 0.037$) and percentage with no health insurance ($\beta = 9.38$, $t = 2.33$, $p = 0.024$).

Interpretation

The omnibus F-tests are all non-significant, meaning the five socioeconomic variables as a group **do not** reliably predict county-level AQI. Adjusted R^2 values below 0.10 across all

three models confirm they explain little of the variance.

As with the hypothesis tests, county-level aggregation is likely masking the relationship between pollution exposure and income that would be more visible at geographic measures.

Overall Analysis

This project investigated whether socioeconomic conditions are associated with air quality outcomes across California counties. To answer this question, we combined geographic visualization, correlation analysis, hypothesis testing, and clustering analysis. Each method provided a different perspective on the relationship between socioeconomic status and environmental quality.

The geographic analysis revealed substantial variation in air quality across California. Rather than being evenly distributed throughout the state, poor air quality was concentrated in a relatively small number of counties. In particular, Fresno, Kern, Los Angeles, Riverside, San Bernardino, and Tulare consistently appeared among the counties with the worst AQI outcomes. This finding suggests that environmental burdens are not shared equally across regions.

Correlation analysis provided additional evidence supporting this pattern. Counties with higher educational attainment and higher household income generally tended to experience better air quality outcomes, while counties with higher poverty and unemployment rates tended to experience worse environmental conditions. Although these relationships do not imply causation, they indicate that socioeconomic status and environmental quality are connected in meaningful ways.

To further investigate this relationship, formal hypothesis tests were conducted comparing lower-income and higher-income counties. The results showed that most AQI measures did not differ significantly between the two groups. This suggests that income alone may not be sufficient to explain air quality disparities across California counties. Because environmental inequality is influenced by multiple factors, clustering analysis was performed using both socioeconomic and environmental indicators. The K-means algorithm identified three distinct county groups. Most notably, one cluster consisting of Fresno, Kern, Los Angeles, Riverside, San Bernardino, and Tulare exhibited substantially worse air quality outcomes than the other counties. These counties also tended to have less favorable socioeconomic characteristics, including lower educational attainment and higher poverty and unemployment rates.

Taken together, the results from all analyses tell a consistent story. Geographic analysis identified where environmental burdens are concentrated, correlation analysis suggested that socioeconomic conditions are associated with environmental outcomes, hypothesis testing demonstrated that income alone cannot fully explain these disparities, and clustering analysis showed that counties experiencing the greatest pollution burdens often share multiple socioeconomic disadvantages simultaneously.

Overall, the findings suggest that environmental burdens in California are associated with broader socioeconomic conditions rather than any single variable. These results highlight the importance of considering multiple dimensions of socioeconomic status when examining environmental inequality and public health outcomes.

Previous studies have used similar environmental and demographic datasets to investigate air pollution disparities across California communities. A study by the Union of Concerned Scientists (UCS) examined inequities in exposure to vehicle-related PM2.5 pollution by combining demographic data from the American Community Survey (ACS) with vehicle emissions estimates from the EPA's MOVES model and PM2.5 exposure estimates generated using the InMAP air quality model. The study found that low-income communities experienced significantly higher levels of PM2.5 pollution exposure than higher-income communities, particularly in Southern California and the Central Valley.

In addition, DeMarsh et al. (2024) conducted an environmental justice analysis of PM2.5 exposure in California's San Joaquin Valley using air quality measurements collected from monitoring and sensor networks together with demographic, socioeconomic, and social vulnerability indicators derived from U.S. Census data. By integrating pollution measurements with community-level demographic characteristics, the study found that disadvantaged communities experienced both higher pollution exposure and greater social vulnerability.

Similarly, the California Air Resources Board (CARB) has performed community-level analyses using multiple public datasets, including air quality monitoring data, emissions inventories, health outcome indicators, demographic variables from the U.S. Census Bureau, and CalEnviroScreen environmental justice metrics. These studies emphasize the importance of combining environmental measurements with socioeconomic characteristics to identify communities that experience disproportionate pollution burdens.

Our project builds directly on this prior work by integrating county-level EPA Air Quality Index (AQI) data with demographic and economic variables from the American Community Survey (ACS DP02 and DP03 datasets). Although our project focuses on county-level AQI trends between 2020 and 2024, the use of similar environmental and demographic datasets allows our findings to be compared with established environmental justice research conducted throughout California.

References:

Papers and Reports:

- DeMarsh, A., et al. (2024). Environmental Justice Analysis of PM2.5 Exposure in California's San Joaquin Valley.
- California Air Resources Board (CARB). (2024). Estimating the Community-Level Health Benefits from Air Pollution Control Programs.
- Reichmuth, D. (2019). Inequitable Exposure to Air Pollution from Vehicles in California. Union of Concerned Scientists.

Datasets and Data Sources Used:

- American Community Survey (ACS). U.S. Census Bureau. https://www.census.gov/programs-surveys/acs
- Air Quality Index (AQI) Data. U.S. Environmental Protection Agency (EPA). https://www.epa.gov/aqs
- Air Quality System (AQS). U.S. Environmental Protection Agency (EPA). https://www.epa.gov/aqs
- CalEnviroScreen. California Office of Environmental Health Hazard Assessment (OEHHA). https://oehha.ca.gov/calenviroscreen
- InMAP (Intervention Model for Air Pollution). Center for Air, Climate, and Energy Solutions. https://inmap.run
- MOVES (Motor Vehicle Emission Simulator). U.S. Environmental Protection Agency (EPA). https://www.epa.gov/moves
- U.S. Census Bureau. Decennial Census and American Community Survey demographic datasets. https://www.census.gov

Potential Limitations & Shortcomings

County-level granularity is too broad to represent wealth:

The analysis runs on 53 counties. However, some countries include a wide disparity in wealth. For example, San Bernardino spans 20 thousand square miles and contains wealthy resort towns as well as poor communities. Averaging income and pollution statistics in this county fails to represent income and pollution in their respective

communities, which can lead to non-significant hypothesis tests and low R^2 values in model summaries.

ACS data spans from 2020 to 2024, while American Community Survey Data is from 2024 Only:

Income and socioeconomic conditions throughout counties change substantially over 4 years, but data is only provided for 2024, while AQI data from 2020 to 2024 is used in our analyses. For the sake of the analyses income and poverty were treated as fixed, so there can be inaccuracies in results of analyses since AQI data is the average of AQI variables from 2020-2024, while income was treated as fixed. Income being fixed is highly unlikely.

Only a subset of California counties have AQI monitoring:

Not every county in California has an EPA air quality monitoring station. Counties without stations are absent from the dataset entirely, introducing a selection bias into the analysis.

COVID-19 affected the 2020 data:

Reduced traffic, reduced industrial activity, and a stay-at-home lifestyle in 2020 produced an extremely low carbon footprint, resulting in clean air across many counties. This unusual activity is an outlier that skews the 2020-2024 averages without any adjustment, which may distort comparisons between income groups.